Ouroboros — CTF Write-up

Challenge author: Dr Programmer
Challenge: Ouroboros | **Category:** Reverse Engineering | **Difficulty:** Medium | **Points:** 200
Prompt: *"Can you escape it?"* | **Flag format:** `r00t{...}`

> **Flag:** `r00t{0ur0b0r0s_6f86002f214455d445fd6522}`

Note: terminal blocks below show the real commands I ran and their real output. `xxd` was not installed in my sandbox, so hex dumps use `od -An -tx1 -v`, which gives the same bytes. Long outputs are trimmed with `...`.

---

1. Recon: what are we given?

The challenge ships a single archive. First move: list it without extracting.

```
┌──(kali㉿kali)-[~/ctf/ouroboros]
└─$ unzip -l ouroboros_chall.zip
Archive:  ouroboros_chall.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
        0  2026-10-21 14:10   ouroboros chall/
    14552  2026-10-04 12:30   ouroboros chall/ouroboros
      734  2026-10-04 12:30   ouroboros chall/isa_outer.txt
    14520  2026-10-04 12:30   ouroboros chall/inner_vm
    14488  2026-10-04 12:30   ouroboros chall/validator
      686  2026-10-04 12:30   ouroboros chall/isa_inner.txt
      499  2026-10-04 12:30   ouroboros chall/outer_disasm.txt
      680  2026-10-04 12:30   ouroboros chall/inner.enc
      263  2026-10-04 12:30   ouroboros chall/outer.orb
    14488  2026-10-04 12:30   ouroboros chall/check
  (__MACOSX junk entries omitted)
```

Extract (ignoring the macOS junk) and identify every file:

```
┌──(kali㉿kali)-[~/ctf/ouroboros]
└─$ unzip -oq ouroboros_chall.zip -x '__MACOSX/*' && cd "ouroboros chall"

┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ file *
check:           ELF 64-bit LSB pie executable, x86-64, dynamically linked,
stripped
inner.enc:       data
inner_vm:        ELF 64-bit LSB pie executable, x86-64, dynamically linked,
stripped
isa_inner.txt:   Unicode text, UTF-8 text
isa_outer.txt:   Unicode text, UTF-8 text
ouroboros:       ELF 64-bit LSB pie executable, x86-64, dynamically linked,
stripped
outer.orb:       data
outer_disasm.txt: Unicode text, UTF-8 text
validator:       ELF 64-bit LSB pie executable, x86-64, dynamically linked,
stripped
```

Four stripped ELF binaries, two opaque data blobs, and three text files. The name "Ouroboros" (a snake eating its own tail) hints at VMs nested inside each other: an **outer VM** that unlocks an **inner VM**.

---

2. Read the documentation

```
┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ cat isa_outer.txt
OUROBOROS OUTER VM — Instruction Set Reference
================================================
Registers: r0-r7 (8-bit)   RAM: 256 bytes   ROM: bytecode (read-only)

Opcode  Mnem   Args          Description
  0x01  LDI    r, imm8       r = imm8
  0x02  XOR    r, r2         r ^= r2
  0x03  ADD    r, r2         r = (r + r2) & 0xFF   [operates on two regs]
  0x04  LDR    r, addr       r = RAM[addr]
  0x05  JNZ    r, offset     if r != 0: PC += offset
  0x06  STR    addr, r       RAM[addr] = r
  0x07  HLT                  halt

Key material: the decryption key for inner.enc is stored at RAM[0x10:0x18].
Usage: ./ouroboros outer.orb inner.enc  (writes inner.orb)
```

```
┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ cat isa_inner.txt
OUROBOROS INNER VM — Instruction Set Reference
================================================
Registers: r0-r7 (8-bit)   RAM: 256 bytes   Stdin: 20 bytes loaded at
RAM[0x00..0x13]

  0x01  LDI    r, imm8       r = imm8
  0x02  XOR    r, r2         r ^= r2
  0x03  ADD    r, imm8       r = (r + imm8) & 0xFF
  0x04  LDR    r, addr       r = RAM[addr]
  0x05  CMP    r, r2         cmp_flag = (r == r2) ? 1 : 0
  0x06  JNZ    offset        if cmp_flag == 1: PC += offset   [JUMP IF EQUAL]
  0x07  HLT    code          exit(code)

Usage: echo -n <20bytes> | ./inner_vm inner.orb
```

Two things stand out. The inner VM wants **20 bytes on stdin**, and its `JNZ` is documented as **jump if equal**, which is easy to misread. The outer VM's key lives at `RAM[0x10:0x18]`.

```
┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ cat outer_disasm.txt
OUROBOROS outer.orb — pre-generated disassembly
Format: ORB1 magic + prog_len=0x0001 + 256-byte RAM init + bytecode

RAM init notable bytes:
  [0x10..0x17]: 3e97a1d3e239c7ba  <- key material (raw1)
  [0x18..0x1F]: f9921126a724c935  <- key material (raw2)

Bytecode (starts at file offset 0x106):
  PC=0000: 07          HLT
```

The outer program is a single `HLT`. All the interesting data is in the RAM image.

---

3. Inspect the raw data files

```
┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ od -An -tx1 -v outer.orb | head -4
 4f 52 42 31 00 01 00 00 00 00 00 00 00 00 00 00
 00 00 00 00 00 00 3e 97 a1 d3 e2 39 c7 ba f9 92
 11 26 a7 24 c9 35 00 00 00 00 00 00 00 00 00 00
 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

`4f 52 42 31` is `ORB1`. RAM starts at file offset 6, so `RAM[0x10]` is at offset `0x16`, which is exactly where `3e 97 a1 d3 ...` sits. The last byte of the file is `07` (`HLT`).

```
┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ od -An -tx1 -v inner.enc | head -3
 12 61 60 50 db 42 c3 d8 5b 2f 32 61 da e0 c3 d8
 5b 2f 32 61 da e0 c3 d8 5b 2f 32 61 da e0 c3 d8
 5b 2f 32 61 da e0 5c 8f 47 f5 1e fa 20 1c e5 99
```

The same 8 bytes (`c3 d8 5b 2f 32 61 da e0`) repeat over and over. That is the signature of a **repeating-key XOR over a mostly-zero file**, since `0x00 ^ key = key`. A second sign that `inner.enc` is simply XOR-encrypted with an 8-byte key.

---

4. Run the outer VM

```
┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ ./ouroboros outer.orb inner.enc
outer VM: OK

┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ od -An -tx1 -v inner.orb | head -3
 49 4e 52 31 01 a2 00 00 00 00 00 00 00 00 00 00
 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 00 00 00 00 00 00 9f 57 1c da 2c 9b fa fc 26 41
```

`49 4e 52 31` is `INR1`, a valid inner-VM header, so the decryption worked. `01 a2` is the program length: 0x01a2 = 418 bytes. File size check: 6 (header) + 256 (RAM) + 418 (code) = **680**, which matches `inner.enc`.

Recovering the key stream

XOR the ciphertext with the plaintext to see what key was actually used:

```
┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ python3 - <<'EOF'
e=open('inner.enc','rb').read(); o=open('inner.orb','rb').read()
ks=bytes(a^b for a,b in zip(e,o))
print(ks[:24].hex(), len(set(ks)))
EOF
5b2f3261dae0c3d85b2f3261dae0c3d85b2f3261dae0c3d8 8
```

Only 8 distinct bytes, so the key is `5b2f3261dae0c3d8`.

Where does that key come from?

It is not either raw half from `outer.orb`. Disassembling the binary shows how it is derived:

```
┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ objdump -d -M intel --no-show-raw-insn ouroboros | sed -n
'/13f3:/,/1498:/p'
    13f3:  movq   xmm0,QWORD PTR [rsp+0x140]     ; RAM[0x18..0x1f]
    13fc:  lea    rcx,[rsp+0x10]
    1403:  movq   xmm1,QWORD PTR [rsp+0x138]     ; RAM[0x10..0x17]
    140c:  pxor   xmm0,xmm1                      ; raw1 ^ raw2
    ...
    1440:  add    edx,0x1
    1449:  and    eax,0x7
    144c:  movzx  eax,BYTE PTR [rsp+rax*1+0x10]
    1451:  ror    al,0x3                         ; rotate right 3
    1454:  xor    al,BYTE PTR [rcx-0x1]          ; ^ current byte
    1457:  xor    eax,0x3c                       ; ^ 0x3c
    ...
    1486:  movzx  edx,BYTE PTR [rsp+rdx*1+0x18]
    148b:  xor    dl,BYTE PTR [rbp+rax*1+0x0]    ; data[i] ^ key[i & 7]
```

So the key is `raw1 ^ raw2` (= `c705b0f5451d0e8f`), mixed through a rotate-right-3, XOR, XOR-`0x3c` pass, giving `5b2f3261dae0c3d8`. That is then XORed repeatedly over the file. This agrees with the key I recovered above.

---

5. Disassemble the inner program

The inner VM has only 7 opcodes, so a disassembler is a few lines of Python. The format is `ID(4) len(2) RAM(256) code(len)`.

```
┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ python3 dis.py | head -30
RAM nonzero: {0x20: 0x9f, 0x21: 0x57, 0x22: 0x1c, 0x23: 0xda, 0x24: 0x2c, ...
0x33: 0x4d,
              0xe0: 0x8a, 0xe1: 0x85, 0xe2: 0xe1, 0xe3: 0xf4, 0xe4: 0x27, ...
0xf3: 0x23}
0000: LDI 06 00          ; r6 = 0
0003: LDI 03 14          ; r3 = 20
0006: LDI 07 01          ; r7 = 1
0009: LDR 00 00          ; r0 = input[0]
000c: LDR 01 20          ; r1 = RAM[0x20]   (table A)
000f: XOR 00 01          ; r0 ^= r1
0012: LDR 02 e0          ; r2 = RAM[0xe0]   (table B)
0015: CMP 00 02          ; flag = (r0 == r2)
```

```
0018: JNZ 03             ; if equal, skip the ADD
001a: ADD 06 01          ; r6 += 1
001d: LDR 00 01          ; ...same block for input[1], RAM[0x21], RAM[0xe1]
...
0185: LDR 00 13          ; last block: input[19]
...
0199: CMP 06 03
019c: JNZ 02
019e: HLT 00
01a0: HLT 01
```

The same 7-instruction block repeats for each of the 20 input bytes. In plain terms:

```
for i in 0..19:
    if (input[i] ^ RAM[0x20+i]) == RAM[0xe0+i]: skip the counter increment
    else: r6 += 1
exit code depends on r6
```

Two 20-byte tables sit in RAM: **A** at `0x20..0x33` and **B** at `0xe0..0xf3`.

---

6. Solve the inner VM, and why it is a trap

The input that satisfies every comparison is `A[i] ^ B[i]`:

```
┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ python3 - <<'EOF'
import subprocess
d=open('inner.orb','rb').read(); ram=d[6:262]
s=bytes(ram[0x20+i]^ram[0xe0+i] for i in range(20))
print(s)
r=subprocess.run(['./inner_vm','inner.orb'],input=s,capture_output=True)
print('match input rc',r.returncode)
r=subprocess.run(['./inner_vm','inner.orb'],input=b'A'*20,capture_output=True
)
print('wrong input rc',r.returncode)
EOF
b'\x15\xd2\xfd.\x0b51\x1a\x82\xa2\x0f\xf4\x9a+\xfd\xb0\xad\xd4\xc2n'
match input rc 0
wrong input rc 1
```

The inner VM accepts that input (exit code 0), but the bytes are **non-printable garbage**, so it cannot be a flag.

To understand the checker, I used the real VM as an oracle, trying all 256 values at each position while the other 19 bytes were wrong:

```
┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ python3 oracle.py
expected b'\x15\xd2\xfd.\x0b51\x1a\x82\xa2\x0f\xf4\x9a+\xfd\xb0\xad\xd4\xc2n'
filler 0 -> rc 1
[[], [], [], [], [], [], [], [], [], [], [], [], [], [], [], [], [], [],
[]]
```

No single correct byte passes on its own: the VM only accepts when **all 20** match. The inner VM is a self-consistent puzzle, but it is a **dead end for the flag**. That fits the *"Can you escape it?"* theme: the nested VMs are a maze.

---

7. Pivot: the other two binaries

The zip also ships `validator` and `check`, which have no obvious use yet. Check what they print:

```
┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ strings -n 3 validator | grep -iE 'correct|fgets|strcmp'
fgets
strcmp
INCORRECT
```

Both read a line with `fgets` and `strcmp` it against a buffer built at runtime. Look at the loop that builds that buffer:

```
┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ objdump -d -M intel --no-show-raw-insn validator | grep -B2 -A6 'xor
eax,0x5c'
    1085:  mov    eax,0x2e
    108a:  lea    rdx,[rip+0xf8f]        # 2020 <.rodata>
    ...
    10e0:  xor    eax,0x5c               ; decode byte with 0x5c
    10e3:  add    rcx,0x1
    10e7:  add    rdx,0x1
    10eb:  mov    BYTE PTR [rcx-0x1],al  ; store decoded byte
    10ee:  movzx  eax,BYTE PTR [rdx]     ; next encoded byte
    10f1:  test   al,al
    10f3:  jne    10e0                   ; until NUL
```

The expected string is stored **XOR** **`0x5c`** at `.rodata+0x20`, starting with `0x2e`. Dump the section:

```
┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ objdump -s -j .rodata validator
Contents of section .rodata:
 2000 01000200 494e434f 52524543 54000000  ....INCORRECT...
 2010 00000000 00000000 00000000 00000000  ................
 2020 2e6c6c28 276c292e 6c3e6c2e 6c2f032b  .ll('l).l>l.l/.+
 2030 2e6c323b 03286c6c 036f682f 2503343d  .l2;.(ll.oh/%.4=
 2040 343d6c6c 2100                        4=ll!.
```

Decode it:

```
┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ python3 - <<'EOF'
d=open('validator','rb').read()
i=d.index(bytes.fromhex('2e6c6c28'))
s=bytearray()
while d[i]!=0: s.append(d[i]^0x5c); i+=1
print(s)
EOF
bytearray(b'r00t{0ur0b0r0s_wr0ng_t00_34sy_haha00}')
```

`r00t{0ur0b0r0s_wr0ng_t00_34sy_haha00}` literally says *"wrong, too easy, haha"*, so it is a **decoy**. Test it:

```
┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ echo 'r00t{0ur0b0r0s_wr0ng_t00_34sy_haha00}' | ./validator
INCORRECT
```

The correct-looking string prints `INCORRECT`. Disassembly confirms why: the `je` on a match jumps to `puts(0x2004)` which is the string `INCORRECT`, and a mismatch prints the string at `0x2006`, which is `CORRECT`. The messages are **swapped**, so `validator` is built to mislead anyone who trusts its output.

---

8. The real flag from `check`

`check` has the same structure, but its `.rodata` string is different:

```
┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ objdump -s -j .rodata check
Contents of section .rodata:
 2000 01000200 494e434f 52524543 54000000  ....INCORRECT...
 2010 00000000 00000000 00000000 00000000  ................
 2020 2e6c6c28 276c292e 6c3e6c2e 6c2f036a  .ll('l).l>l.l/.j
 2030 3a646a6c 6c6e3a6e 6d686869 69386868  :djlln:nmhhii8hh
 2040 693a386a 696e6e21 00                 i:8jinn!.
```

The first 15 bytes are the same as `validator`'s (so the prefix `r00t{0ur0b0r0s_` repeats) and then it diverges. Same decode, `^ 0x5c`:

```
┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ python3 - <<'EOF'
import subprocess
d=open('check','rb').read()
i=d.index(bytes.fromhex('2e6c6c28'))
s=bytearray()
while d[i]: s.append(d[i]^0x5c); i+=1
print(s.decode())
r=subprocess.run(['./check'],input=bytes(s)+b'\n',capture_output=True)
print(r.stdout)
EOF
r00t{0ur0b0r0s_6f86002f214455d445fd6522}
b'CORRECT\n'
```

Feeding the decoded string back into `check` prints **CORRECT**.

```
┌──(kali㉿kali)-[~/ctf/ouroboros/ouroboros chall]
└─$ echo 'r00t{0ur0b0r0s_6f86002f214455d445fd6522}' | ./check
CORRECT
```

---

9. Flag

```
r00t{0ur0b0r0s_6f86002f214455d445fd6522}
```

---

10. Honest notes and open questions

- I could not submit the flag to the platform from my sandbox, so the platform's verdict is the final word.
- The outer VM's program is only `HLT`. I did not find what a non-trivial outer program was supposed to do, so the inner VM may be a red herring. If the author intended the outer program to alter the key material, the intended `inner.orb` could differ from the one I produced.
- I did not fully work out the inner VM's final counter check. Black-box testing showed that it accepts only the exact 20-byte input and rejects anything else, which was enough to conclude it is not the path to the flag.

---

11. Key takeaways

1. **Read the docs first, but do not trust them blindly.** The ISA files were accurate and useful (key location, `JNZ` = jump-if-equal), yet two of the shipped binaries were built to mislead.
2. **Repeating bytes in an encrypted file reveal the key.** A repeating 8-byte pattern in `inner.enc` meant repeating-key XOR over zero-padding, so XORing ciphertext with the decrypted output gave the key directly.
3. **Static analysis of the tool that decrypts is worth the time.** Disassembling `ouroboros` explained *how* the key is derived (XOR of the two halves, rotate, XOR) instead of just trusting its output.
4. **A correct solution can still be a dead end.** The inner VM was fully solvable but produced non-printable bytes. If the output is not shaped like a flag, keep looking at what else the challenge ships.

5. **Decoys exist; check the shape of what you find.** `validator` contained a taunting fake flag and swapped its `CORRECT` / `INCORRECT` messages. Do not trust a program's verdict until you have read the code that prints it.
6. **Single-byte XOR with a constant (here `0x5c`) is a classic string-hiding trick.** When `strings` shows nothing useful but the code has a tight `xor reg, imm8` loop over `.rodata`, decode the data yourself.
7. **Use the binary as an oracle, but know its limits.** Testing every byte value per position is a fast black-box technique. Here it also told me something useful: the check is all-or-nothing, so there is no per-byte side channel.